Pyroclast: GPU-Accelerated Monte Carlo Simulation for Lava-Flow Habitat Impact Assessment

Emanuele Galiano

May 26, 2026

Contents

Chapter 1

Kernel Monte Carlo

In this chapter we present the design and implementation of the kernel Monte Carlo method used in Pyroclast for simulating lava flow and assessing habitat impact. In the first section we discuss the mathematical formulation...

1.1 Mathematical Formulation

The problem consists of estimating the probability that a lava flow will cause the destruction of a specific habitat area, given a pre-calculated map of invasion probabilities.

1.1.1 Variables and Parameters

We can define the variables of the problem as follows:

- Let N_c be the total number of active cells that make up the habitat under evaluation.
- Let $\mathcal{Q} = \{p_0, p_1, \ldots, p_{N_c-1}\}$ be the set of invasion probabilities for each cell of the habitat, where p_i is the probability that the lava flow will invade cell i.
- Let $\theta \in [0, 1]$ be the critical fraction or destruction threshold, which represents the minimum fraction of the habitat that must be invaded for it to be considered destroyed.
- Let R be the total number of Monte Carlo simulations to be performed.

1.1.2 The Monte Carlo Run

For a single simulation identified by the index r,where $0 \leq r < R$, we perform the following steps:

1. **Sampling**: For each individual cell k of the habitat, we draw a pseudo-random number $X_{r,k} \sim \mathcal{U}(0, 1)$.
2. **Invasion Evaluation**: Cell k is considered "invaded" by lava in simulation r if the draw value is less than or equal to its invasion probability, i.e., if $X_{r,k} \leq p_k$, and it can be formalized with an indicator function

$$I_{r,k} = \begin{cases} 1 & \text{if } X_{r,k} \leq p_k \\ 0 & \text{otherwise} \end{cases} \tag{1.1}$$

3. **Destruction Assessment**: We first define the total number of invaded cells in simulation r as

$$C_r = \sum_{k=0}^{(N_c-1)} I_{r,k} \tag{1.2}$$

and then the habitat is considered destroyed in run r if the fraction of invaded cells exceeds the critical threshold θ (normalized by the total number of cells):

$$D_r = \begin{cases} 1 & \text{if } \dfrac{C_r}{N_c} \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{1.3}$$

1.1.3 Monte Carlo Estimator

The ultimate goal of the parallel exeuction is to compute the total number of simulations that resulted in habitat destruction. This requires summing D_R, over all R runs. The Monte Carlo estimator for the probability of habitat destruction, also the overall probability of destruction $\hat{P}_{\text{destruction}}$, can be estimated as:

$$\hat{P}_{\text{destruction}} = \frac{1}{R} \sum_{r=0}^{R-1} D_r \tag{1.4}$$

1.2 Standard Kernel

The first version of the Monte Carlo kernel implemented in Pyroclast is a straightforward parallelization of the algorithm described in the previous section. Each simulation run is executed independently, and the results are aggregated by a reduction operation at the end of the kernel execution.

1.2.1 Kernel Design

The kernel operates on a 1-dimensional grid of work-items, where each work-item is reponsible for executing a whole Monte Carlo simulation on the full array of invasion probabilities.

Work-Item Mapping. A critical architectural decision in this baseline was how to map the computational workload to the GPU's work-items. The most intuitive approach is the pure stream processing model, where each work-item is responsible for executing only single Monte Carlo. This approach was delibartely discarded due to a sever hardware inefficiency known as the "prime number problem"[1]: in OpenCL the number of launched work-items is defined by the Global Work Size, which is hwardware-partitioned into smaller blocks called work-groups, defined by the Local Work Size. If we tie the number of work-items to the number of simulations, an user requesting a number of simulations that is prime would force a primw GWS and since the only integer divisors of a prime number are 1 and itself, the hardware would be forced to launch work-groups containing a single work-item (LWS = 1).

[1] Also knowns as one of the "leaky abstractions" of GPU programming.

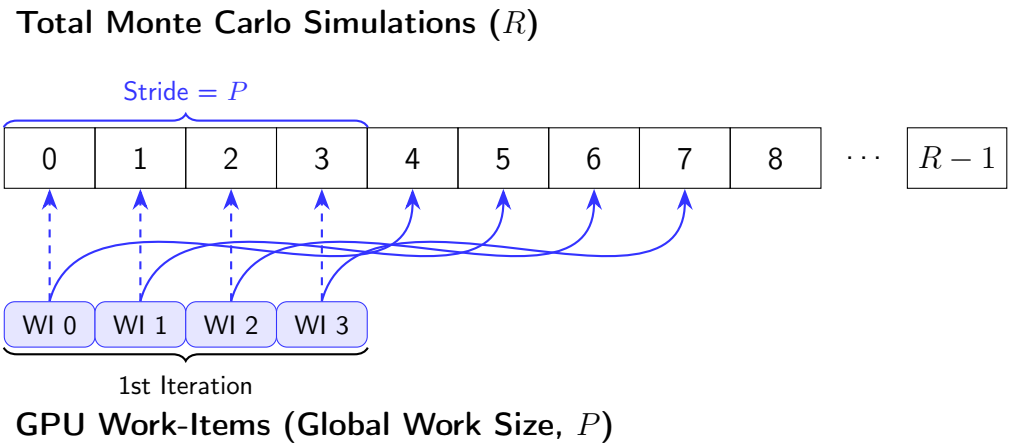

Figure 1.1: Visual representation of the Grid-Stride Loop (Sliding Window) approach. A hardware-optimized grid of P work-items iteratively processes the total R simulations by advancing with a stride equal to the Global Work Size.

Sliding Window. To resolve this limitation a sliding window approach is adopted. Instead of launching a grid sized perfectly to the dataset, we launch a hardware optimized grid and allow each work-item to process multiple runs through a for loop, taking strides[2] equal to the entire GWS.

1.2.2 Algorithmic Complexity

The theoretical time complexity of the kernel depends on the total number of Monte Carlo simulations R, the number of cells N_c, the number of work-items P launched and the dimension of the work-groups L (LWS).

Time Complexity. THe time complexity of the kernel can be divided in due principal components:

Sampling and Monte Carlo Run: Kernel doesn't map a single work-item, so the for loop iterates over runs for a total of R/P iterations. For each run, the kernel iterates over the N_c cells of the habitat in order to evaluate the invasion, resulting in a time complexity

$$\mathcal{O}\left(\frac{R}{P} \cdot N_c\right)$$

Reduction: After all work-items have completed their assigned runs, a reduction operation is performed to sum the results of all simulations. The tree reduction operates on an array in local memory of size L (the number of work-items in a work-group) and since LWS is a power of 2, the reduction divides the problem size by 2 at each step, resulting in a time complexity of

$$\mathcal{O}(\log_2 L)$$

[2]The stride represents the number of simulations processed by each work-item in the sliding window approach.

Finally, the overall time complexity of the kernel can be expressed as:

$$T(N_c, R, P, L) = \mathcal{O}\left(\frac{R}{P} \cdot N_c + \log_2 L\right) \tag{1.5}$$

Space Complexity. The space complexity of the kernel is determined in hierarchies:

- **Global Memory**: The kernel reads the compacted array of probabilities `p_vec`, which has a size of $\mathcal{O}(N_c)$. In output, it writes a partial sum of the destruction results not for each work-item, but for each work-group. Assuming a global work size of P and a local work size of L, the output array `partial` has a size of P/L. Thus, the total global memory complexity is $\mathcal{O}(N_c + P/L)$.
- **Local Memory**: Each work-group allocates a shared `scratch` array used for the parallel tree reduction. The size of this array corresponds exactly to the local work size L (which is statically set to 256). Therefore, the local memory complexity is $\mathcal{O}(L)$ per work-group.
- **Private Memory**: Each individual work-item maintains only a few local state variables (such as the `private_sum` accumulator, loop counters, and thread IDs). This requires a strictly constant amount of memory, resulting in a private space complexity of $\mathcal{O}(1)$ per work-item.

1.3 1D Ping-pong Kernel

While the standard 1D kernel performs an efficient parallel tree reduction, it updates the shsared memory array in-place. This causes the use of two barriers:

- One before writing in local memory, to ensure that all work-items have completed their computations and are ready to write their partial sums.
- One after writing, to ensure that all work-items have completed their writes before any work-item can read from the shared memory for the reduction step.

1.3.1 Double buffering

In order to eliminate the need for the second barrier, a double buffering technique is adopted. Instead of using a single shared memory array for the reduction, two separate arrays are allocated: one for reading and one for writing. During each step of the reduction, work-items read from one buffer and write their results to the other buffer. After each step, the roles of the buffers are swapped. This approach allows work-items to proceed with their computations without waiting for all other work-items to complete their writes, thus reducing synchronization overhead.

1.3.2 Trade-offs

The implementation of this kernel explicitly highlights the trade-off between memory usage and synchronization overhead.

Benefits. By garanteeing that the buffer being read from the source is physically distinct from the buffer being written to the destination, we can eliminate any possibility of Write-After-Read hazards. This allows the step to be safely executed with only a single barrier at the top of the loop to publish the initial writes, potentially allowing work-items to proceed with their computations without waiting for all other work-items to complete their writes, thus reducing synchronization overhead.

The cost. To achieve this safety, the kernel requires an additional local array. This part we'll be seen in the space complexity analysis at 1.3.3, but it is important to note that this additional memory usage may limit the maximum local work size that can be used, as the total local memory available on the GPU is a fixed resource. This could potentially reduce the degree of parallelism and impact performance if the local work size needs to be reduced to accommodate the additional buffer.

1.3.3 Algorithmic Complexity

The theoretical time complexity of the 1D Ping-Pong kernel remains fundamentally similar to the standard version, as it depends on the total number of Monte Carlo simulations R, the number of cells N_c, the number of launched work-items P, and the dimension of the work-groups L (LWS).

Time Complexity. The time complexity of the kernel is divided into the same two principal components as the standard kernel and the time complexity of each component remains unchanged:

$$T(N_c, R, P, L) = \mathcal{O}\left(\frac{R}{P} \cdot N_c + \log_2 L\right) \tag{1.6}$$

Space Complexity. The space complexity highlights the main architectural trade-off of the double-buffering approach:

- **Global Memory**: The kernel reads the compacted `p_vec` array of size $\mathcal{O}(N_c)$ and writes to the `partial` array of size P/L. The total global memory complexity remains $\mathcal{O}(N_c + P/L)$.
- **Local Memory**: To eliminate synchronization hazards with fewer barriers, each work-group allocates *two* shared arrays instead of one: a primary `scratch1` array of size L and a secondary `scratch2` array of size $L/2$. While asymptotically this is still $\mathcal{O}(L)$ per work-group, the absolute footprint is strictly $1.5 \times L$. This increased local memory usage directly reduces the maximum hardware occupancy (number of active work-groups) on the Compute Units.
- **Private Memory**: Each work-item maintains the usual local state variables alongside a new `val` register used to carry the running sum across loop iterations safely. This still requires a constant amount of memory, resulting in a private space complexity of $\mathcal{O}(1)$ per work-item.

ù

1.4 Vectorized Kernels

One of the ideas of this project was to explore the potential performance benefits of vectorization in the Monte Carlo kernel. By leveraging the SIMD (Single Instruction, Multiple Data) capabilities of modern GPUs, we can process multiple data elements in parallel within a single work-item, potentially improving throughput and reducing execution time.

1.4.1 Vectorized Random Number Generation

The transition from a scalar to a vectorized Monte Carlo kernel requires a structural adaptation in the generation of pseudo-random numbers. As implemented in the `misc_vec.h` header and its underlying OpenCL definitions, the vectorized approach replaces scalar state variables with native OpenCL vector types (e.g., `uint2`, `uint4`, or `uint8`). Instead of generating a single random number per step, the generator advances `VEC_WIDTH` independent lanes concurrently.

By advancing the MWC64X generator with bitwise and arithmetic operations directly on vector types, the OpenCL compiler can map these calculations to single hardware vector instructions, thereby increasing the Instruction-Level Parallelism. Furthermore, processing a block of `VEC_WIDTH` cells within a single loop iteration significantly increases the ratio of useful arithmetic work to loop-control overhead, maximizing the overall throughput (i.e., evaluated elements per second).

Stream Independence and Seeding. A critical requirement for the validity of the Monte Carlo method is ensuring that no two work-items—and no two vector lanes within a single work-item—sample the same sequence of pseudo-random numbers. Is guaranteed this stream independence, by the use of an external library, at two distinct levels:

Inter-thread Independence: As for the baseline standard kernel, each simulation run relies on a `base_offset` provided by the host. The starting point in the stream is calculated as a function of the global simulation ID (r), ensuring that each run operates on a mathematically disjoint segment of the generator's 2^{63} period.

Intra-thread (Lane) Independence: When using a vector state (e.g., `uint4`), the internal structure must maintain independent seeds for each of its channels. The specific initialization functions (such as `MWC64XVEC4_SeedStreams`) do not simply duplicate the same seed across the vector. Instead, they apply a deterministic mathematical skip, assigning a unique internal offset (ranging from 0 to `VEC_WIDTH` $- 1$) to each individual lane.

1.4.2 Trade-offs and Statistical Robustness

The vectorized kernel design offers significant performance advantages by maximizing the use of SIMD capabilities. However, this approach also introduces a trade-off in terms of the statistical properties of the generated random numbers. Since each lane in the vectorized generator is initialized with a different offset, the sequence of random numbers produced by each lane will differ from the sequence that would be generated in a scalar implementation. This means that while the vectorized kernel maintains statistical robustness (i.e., it produces statistically valid random numbers), it does not produce the same sequence of random numbers as the scalar version for a given seed.

Z-test. To mitigate this issue, we can perform a proportion Z-test[3] on the outputs of the scalar and vectorized kernels. We define:

- H_0: The probability of habitat destruction estimated by the scalar kernel is equal to the probability estimated by the vectorized kernel. That means

$$\hat{P}_{\text{destruction, scalar}} = \hat{P}_{\text{destruction, vectorized}}$$

- H_1: The probability of habitat destruction estimated by the scalar kernel is not equal to the probability estimated by the vectorized kernel. That means

$$\hat{P}_{\text{destruction, scalar}} \neq \hat{P}_{\text{destruction, vectorized}}$$

To perform the test, we need to collect the following data:

- **Simulations**: We run R simulation on both kernels and record the number of simulations that resulted in habitat destruction for each kernel, denoted as D_{scalar} and $D_{\text{vectorized}}$ respectively.
- **Significance Level**: We choose a significance level α (commonly set to 0.05) to determine the threshold for rejecting the null hypothesis. If the p-value of the test is less than α, we reject H_0 and conclude that there is a statistically significant difference between the two kernels. Otherwise, we fail to reject H_0 and conclude that there is no statistically significant difference in the estimated probabilities of habitat destruction between the scalar and vectorized implementations.

TODO: insert figure

Chi-square test. Alternatively, we can perform a chi-square test for independence on the contingency table of outcomes (destruction vs. no destruction) for both kernels. This test will allow us to determine if there is a significant association between the type of kernel used and the outcome of the simulations. The null hypothesis H_0 would state that the kernel type and the outcome are independent, while the alternative hypothesis H_1 would state that there is a dependence between the kernel type and the outcome.

TODO: insert figure

1.4.3 Algorithmic Complexity

The theoretical time and space complexity of the vectorized kernel introduces the vector width W (defined as `VEC_WIDTH`) as a new fundamental hardware parameter. While the high-level grid-stride loop and tree reduction structures remain identical to the scalar baselines, the per-cell workload is drastically altered.

Time Complexity. The time complexity can still be divided into the two principal components (Monte Carlo sampling and Local Memory reduction). However, the sampling phase is now accelerated by the SIMD execution:

[3] A common statistical test used to compare the proportions of two independent samples.

Sampling and Monte Carlo Run: The kernel still iterates over the runs with a total of R/P iterations per work-item. However, within each run, the inner loop no longer iterates exactly N_c times. Instead, the habitat is processed in chunks of size W, resulting in $G = \lceil N_c/W \rceil$ iterations. In each iteration, a single OpenCL vector instruction processes W cells simultaneously.

Reduction: The local memory tree reduction remains strictly unchanged, operating over the L work-items of the work-group, yielding $\mathcal{O}(\log_2 L)$.

The overall time complexity of the vectorized kernel is therefore expressed as:

$$T(N_c, R, P, L, W) = \mathcal{O}\left(\frac{R}{P} \cdot \frac{N_c}{W} + \log_2 L\right) \tag{1.7}$$

Strictly speaking in terms of Big-$\mathcal{O}$ asymptotic notation, W is a hardware constant, making this mathematically equivalent to Equation 1.5. In GPGPU practice, however, the explicit division by W represents a tangible reduction in loop-control overhead and a massive increase in actual memory throughput via wider coalesced loads (`vload`W).

Space Complexity. The space complexity highlights the critical trade-off of the vectorized approach, specifically regarding the GPU registers allocation:

- **Global Memory**: The `p_vec` array requires a slight ceiling padding to ensure its size is a multiple of W, preventing out-of-bounds memory accesses during the final `vload`W. The complexity remains identically $\mathcal{O}(N_c + P/L)$.
- **Local Memory**: The vectorized sampling can be seamlessly plugged into either the standard or the double-buffering (ping-pong) reduction shells. Thus, the local memory footprint is independent of W and remains either $\mathcal{O}(L)$ or $\mathcal{O}(1.5 \times L)$ per work-group, depending on the chosen reduction adapter.
- **Private Memory (Register Pressure)**: This is where the vectorized kernel differs most. Instead of maintaining a few scalar state variables, each work-item must allocate hardware registers to hold vector types: a W-wide state for the RNG (`VEC_STATE_T`), a W-wide accumulator (`INTV acc`), and W-wide buffers for probabilities and drawn floats. Consequently, the private memory footprint scales linearly with the vector width, becoming $\mathcal{O}(W)$ per work-item. High register pressure is a well-known OpenCL constraint: if a single work-item requests too many registers (e.g., when $W = 8$ or $W = 16$), the Compute Unit is forced to schedule fewer concurrent wavefronts/warps, directly decreasing the hardware occupancy and potentially negating the SIMD speedup.

1.5 Transitioning into 2D Topologies

Every kernel presented so far shares the same skeleton: a one-dimensional grid is launched over the simulation axis R, and each work-item walks the N_c cells of the habitat *sequentially* to compute the invaded count C_r of Section 1.1.2. This is visible in the recurring sampling term $\mathcal{O}\left(\frac{R}{P} \cdot N_c\right)$ of every complexity analysis: the factor N_c is a serial inner loop that no kernel has yet broken. The vectorized kernel of Section 1.4 was the first attempt to attack it, but it did so *within* a single work-item, widening each step from one cell to

W cells through SIMD registers; the loop itself stayed sequential and the cost was paid in register pressure.

The natural next step is to stop hiding the cell index inside a loop and promote it to a genuine *second grid dimension*. If the run index r lives on one axis and the cell index k on another, then the sum $C_r = \sum_k I_{r,k}$ is no longer a serial scan performed by one thread, but a quantity computed cooperatively by a whole row of threads and then collapsed by a parallel reduction. The destruction assessment D_r and its accumulation over runs become a *second* reduction along the orthogonal axis. This is the idea behind the 2D kernels: a Monte Carlo run is decomposed onto a two-dimensional tile of work-items, and the algorithm closes with two reductions instead of one.

1.5.1 Kernel Design

The kernel is launched on a two-dimensional `NDRange`. We call the work-items along the fast axis (`dim0`) the *cell lanes* and denote their number by L_c; the work-items along the slow axis (`dim1`) are the *run lanes*, denoted R_r. A work-group is therefore an $L_c \times R_r$ tile, and its size is $L = L_c \cdot R_r$. The cell axis is sized so that a single work-group spans it entirely (the global size on `dim0` equals L_c), which keeps the cell reduction local to the work-group; the run axis is tiled across many work-groups and, exactly as in the baseline, traversed by a grid-stride loop so that each run lane processes R/P_r runs, where P_r is the global size on the run axis.

Interleaved Cell Partition. How the N_c cells are split among the L_c cell lanes is the decisive design choice, because it dictates the memory access pattern. We adopt an *interleaved* partition: cell lane L is responsible for the cells L, $L + L_c$, $L + 2L_c, \ldots$ This is the same sliding window-L_c sweep that the baseline used over the run axis (Figure 1.1), only applied to the cells, and it is chosen precisely because at every step the L_c lanes of a row touch L_c *consecutive* addresses of `p_vec` (Figure 1.2). Consecutive work-items hitting consecutive addresses is the textbook condition for a coalesced global load.

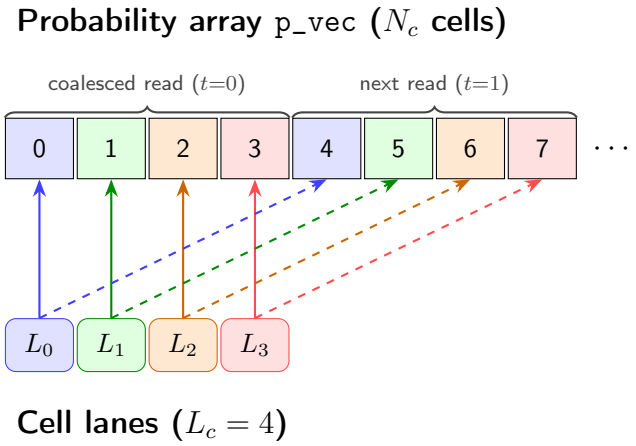

Figure 1.2: Interleaved cell partition with $L_c = 4$ cell lanes, colour-coded by owner. Lane L owns the cells of its colour $(L, L + L_c, \ldots)$: the solid arrow is the read at step $t = 0$, the dashed arrow the read at step $t = 1$, one stride of L_c later. At every step the four lanes touch four consecutive cells of `p_vec`, so the global loads are coalesced.

This partition also fixes the structure of the random-number stream. To keep the lanes

independent while letting each one draw its samples sequentially, cell lane L seeds the MWC64X generator once at the stream position `run_base`$+L\cdot G$ and draws $G = \lceil N_c/L_c \rceil$ samples, where a run owns the contiguous segment of length `run_stride` $= G \cdot L_c$. No two lanes (and no two runs) ever share a stream position, so statistical independence is preserved exactly as before. The reader will recognise this as *the very same stream layout* as the vectorized kernel with $W = L_c$: the cells of a run are grouped into blocks of L_c and assigned across lanes, only here the "lanes" are real work-items rather than SIMD channels. As a consequence the 2D kernel is bit-exact with the vectorized kernel of equal width, but it is *not* bit-exact with the scalar baseline; its correctness is therefore validated statistically through the same proportion Z-test of Section 1.4.2. As in the vectorized case, `p_vec` is padded up to `run_stride` with a -1.0 sentinel so that the trailing samples ($k \geq N_c$) satisfy $x \leq -1.0$ trivially and never count as invaded, removing any bounds check from the hot loop.

TODO: insert figure

The Two Reductions. Once every cell lane has finished its strided scan, the run is not yet resolved: each lane holds only a *partial* invaded count over the cells it owns, and these partials must be combined. The kernel does so with two reductions acting on the two orthogonal axes of the tile (Figure 1.3).

Reduction 1 — over the cell lanes. The L_c partials of a single run row are summed into that run's total invaded count $C_r = \sum_k I_{r,k}$. Each lane writes its partial into a contiguous per-row slice of the local `scratch` buffer, and the slice is collapsed by the usual power-of-two tree: at each of the $\log_2 L_c$ steps the active lanes halve and every survivor adds the value of its partner, until lane 0 of the row holds C_r. That lane immediately applies the strict-threshold rule of Section 1.1.2, $D_r = \mathbb{1}[\, C_r/N_c > \theta \,]$, and adds the resulting 0/1 flag to a private accumulator that persists across the R/P_r runs the row visits through the grid-stride loop. Reduction 1 is therefore performed once *per run*, inside the run loop.

Reduction 2 — over the run lanes. When the run loop ends, each of the R_r rows has accumulated, in its own lane 0, the number of destruction events that row witnessed. A second power-of-two tree — this time along the run axis, in $\log_2 R_r$ steps — sums these R_r counts into a single integer: the destruction count of the whole work-group. Work-item $(0, 0)$ writes it to `partial`$[g]$. Exactly as for the one-dimensional kernels, the host then folds the n_{wg} work-group partials into the final count with the shared `reduce_sum_int` reducer and divides by R to obtain $\hat{P}_{\text{destruction}}$.

The point of the construction is that the serial inner sum $C_r = \sum_k I_{r,k}$ — an $\mathcal{O}(N_c)$ loop buried inside every work-item of the one-dimensional kernels — has been replaced by the cooperative Reduction 1 of depth $\mathcal{O}(\log_2 L_c)$. This is precisely the inner loop the 2D topology set out to parallelise.

Transposed Launch Grid. Because the two reductions act on orthogonal axes, an obvious question is whether the assignment of those axes to the hardware dimensions matters. To answer it we implement a second kernel that is simply the *transpose* of the first: the run lanes are placed on the fast axis `dim0` and the cell lanes on `dim1`, leaving the computation and the partition of work untouched. Since the mapping $(r, k) \mapsto$ stream position is unchanged and integer addition is associative, the transposed kernel returns a result that is *bit-exact* with its non-transposed sibling; what differs is only the memory behaviour of the sampling and of Reduction 1, examined next. The transposed variant

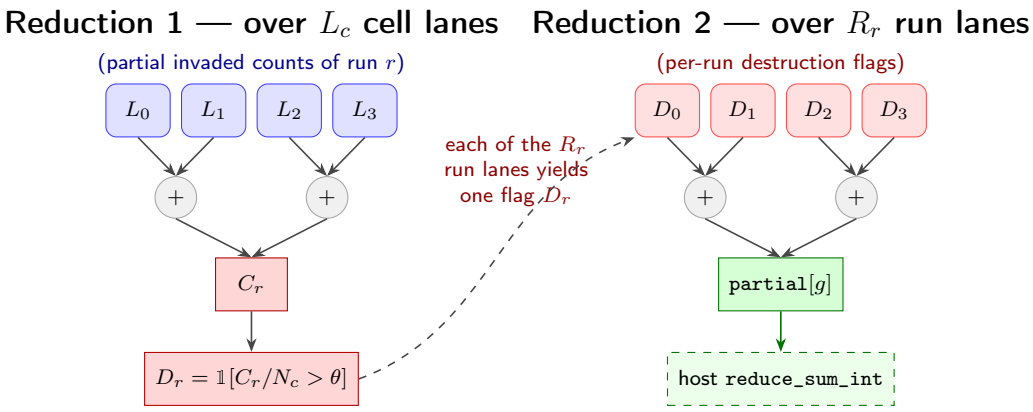

Figure 1.3: The two reductions, shown as binary trees ($L_c = R_r = 4$). **Reduction 1** (left) sums the L_c cell-lane partials of one run r into the invaded count C_r in $\log_2 L_c$ steps, then lane 0 derives the destruction flag D_r. Each of the R_r run lanes produces one such flag; these are the leaves of **Reduction 2** (right), which sums them in $\log_2 R_r$ steps into the work-group total `partial`$[g]$. The host finally folds the n_{wg} work-group partials with `reduce_sum_int`.

therefore exists not to be faster, but to *isolate and measure* the cost of the launch-grid mapping, all other factors being identical.

Coalescing and Bank Conflicts. Two distinct memory effects separate the two launch grids, and both favour the non-transposed kernel. They concern, respectively, the global reads of `p_vec` (which dominate the kernel's global traffic) and the local-memory accesses of the cell reduction.

Global-memory coalescing. On a GPU the work-items of a warp issue their global loads together, and the hardware is fastest when those loads fall on consecutive addresses, which it can then service with a single *coalesced* transaction. In the non-transposed grid the cell lanes occupy the fast dimension, so a warp carries consecutive values of L and, at step t, reads the consecutive addresses `p_vec`$[\,tL_c + L\,]$: a textbook coalesced load (this is exactly the alignment depicted in Figure 1.2). In the transposed grid the fast dimension is the run axis, so a warp holds a *fixed* cell index L and varying run lanes; all of its work-items request the *same* address. The hardware serves this as a broadcast — efficient in isolation, but it no longer streams a contiguous span of `p_vec` across the warp, forfeiting the wide-load advantage.

Local-memory bank conflicts. Shared (local) memory is split into 32 banks, and a warp completes a local access in one cycle only if its threads hit 32 distinct banks; threads landing on the same bank are serialised. The non-transposed kernel stores a run's partials in a contiguous slice `scratch`$[\,\rho \cdot L_c + L\,]$, so the threads combined at each step of Reduction 1 are one word apart (stride 1) and necessarily map to distinct banks — the reduction is conflict-free, by the same argument as the map-centric layout of Section 1.6.1. The transposed kernel indexes the very same data as `scratch`$[\,L \cdot R_r + \rho\,]$, so the partners of a reduction step lie R_r words apart; since R_r is a power of two it shares its factors with the 32 banks, several threads of a warp collide on the same bank, and each reduction step is serialised.

Neither penalty is catastrophic in practice: `p_vec` is only a few hundred kilobytes and resides comfortably in the L2 cache, which absorbs much of the difference in the global-

read pattern, while the cell reduction is a small $\log_2 L_c$ fraction of the kernel's work. The transposed kernel is therefore slower than the non-transposed one but not dramatically so; the benchmark of Section 1.7 quantifies the residual gap.

1.5.2 Algorithmic Complexity

The complexity is expressed in terms of the two new structural parameters L_c and R_r (with $L = L_c \cdot R_r$), the run-axis global size P_r, and the familiar R and N_c.

Time Complexity. The sampling phase is now shared by the cell lanes, while the closing of a run involves the two reductions:

- **Sampling and Monte Carlo Run**: each run lane processes R/P_r runs; within a run, the N_c cells are split across the L_c lanes, so each lane draws only $G = \lceil N_c/L_c \rceil$ samples. The sampling work per run lane is therefore $\mathcal{O}\left(\frac{R}{P_r} \cdot \frac{N_c}{L_c}\right)$.
- **Reduction 1 (cells):** performed once per run inside the grid-stride loop, the cell-lane tree reduction costs $\mathcal{O}(\log_2 L_c)$, for a total of $\mathcal{O}\left(\frac{R}{P_r} \log_2 L_c\right)$.
- **Reduction 2 (runs):** performed once at the end, the run-lane tree reduction costs $\mathcal{O}(\log_2 R_r)$.

The overall time complexity is thus

$$T(N_c, R, P_r, L_c, R_r) = \mathcal{O}\left(\frac{R}{P_r}\left(\frac{N_c}{L_c} + \log_2 L_c\right) + \log_2 R_r\right) \tag{1.8}$$

A subtlety deserves emphasis. If the total number of work-items $P = L_c \cdot P_r$ is held equal to that of the one-dimensional launch, the sampling term reduces to $\frac{R}{P_r} \cdot \frac{N_c}{L_c} = \frac{R}{P} \cdot N_c$, i.e. exactly the cost of Equation 1.5: the same $R \cdot N_c$ samples are simply spread over the same number of threads. In pure Big-$\mathcal{O}$ terms the 2D kernel is therefore not cheaper—if anything the extra $\frac{R}{P_r} \log_2 L_c$ of the per-run reduction makes it marginally heavier. As with vectorization, the benefit is architectural rather than asymptotic: the long serial inner scan of length N_c is replaced by a shallow $\log_2 L_c$ tree, and the cell lanes now issue wide coalesced loads instead of the strided single-cell accesses of the baseline.

Space Complexity. The space hierarchy reveals where the 2D approach pays, and where it economises with respect to the vectorized kernel:

- **Global Memory**: the kernel reads `p_vec` (padded up to a multiple of L_c) of size $\mathcal{O}(N_c)$, and writes one integer per work-group, i.e. an output array of size $n_{wg} = P_r/R_r$. The total global memory complexity is $\mathcal{O}(N_c + P_r/R_r)$.
- **Local Memory**: a single dynamic `scratch` buffer serves both reductions: $L_c \cdot R_r$ integers for the cell reduction plus a further R_r for the run reduction, i.e. $L + R_r$ integers, which is $\mathcal{O}(L)$ per work-group—the same order as the standard kernel.
- **Private Memory**: each work-item keeps only its own partial invaded count, a destruction accumulator, the RNG state and a handful of loop counters, for a strictly constant $\mathcal{O}(1)$ footprint. This is the key contrast with the vectorized kernel: the

2D design achieves cell-level parallelism by distributing the work across *work-items and local memory* rather than across *vector registers*, so it avoids the $\mathcal{O}(W)$ register pressure that throttles occupancy at large widths—trading it instead for the local-memory traffic and the barriers of the two reductions.

1.6 Kernel Map-Centric

In addition to the standard habitat-centric kernel design, we also explored a map-centric approach. All the kernels presented so far are *habitat-centric*: each launch operates on a single habitat through its stream-compacted probability vector `p_vec`, sampling only the cells that belong to that habitat. When several habitats overlap geographically, however, the same map cell belongs to more than one habitat, and the habitat-centric design re-samples it—drawing a fresh pseudo-random number—once per overlapping habitat. The *map-centric* kernel inverts this mapping: it sweeps the entire geographic map a single time, draws exactly one sample per cell, and propagates the outcome to every habitat that occupies that cell in the same pass. Overlap therefore costs no additional random numbers.

1.6.1 Kernel Design

The kernel is launched on the same 1-dimensional grid of work-items over the simulation axis R as the baseline, and reuses the grid-stride loop described in Section 1.2. The fundamental difference lies in what a single run iterates over: instead of the N_c cells of one habitat, a run now sweeps the M cells of the whole map (more precisely, of the union footprint of the habitats in the current batch) and updates all of them simultaneously.

Bitmask Encoding. For each map cell k the host provides a 64-bit presence bitmask `habitat_mask[k]`, in which bit h is set if and only if habitat h of the current batch occupies cell k. Because a `ulong` holds exactly 64 bits, at most `MAX_BATCH_SIZE` $= 64$ habitats can be encoded per launch; the host partitions any larger problem into batches of at most 64 habitats. This compact encoding is what allows a single cell sweep to serve every habitat at once.

Single-Sweep Sampling and RNG Reuse. For each run r the MWC64X generator is seeded exactly once, at the stream position `base_offset` $+ r \cdot M$, so that run r owns the contiguous, non-overlapping stream segment $[$`base_offset` $+ r \cdot M$, `base_offset` $+ (r+1) \cdot M$ $)$. The work-item then sweeps all M cells: a cell whose mask is zero (no habitat present) is skipped outright, while for every non-empty cell a single `Step()` produces a draw $x_k \sim \mathcal{U}(0,1)$. If $x_k \leq p_k$ the cell is credited to all the habitats present in it. Crucially, the per-cell RNG cost is identical to the habitat-centric baseline (one draw per cell), so the saving from overlap reuse is genuine: a cell shared by ten habitats is sampled once instead of ten times, while stream independence across runs is preserved exactly as before.

Branchless Accumulation. Once a cell is found to be invaded, the kernel distributes the outcome to the habitats through the inner update

```
run_invaded[h] += (mask >> h) & 1
```

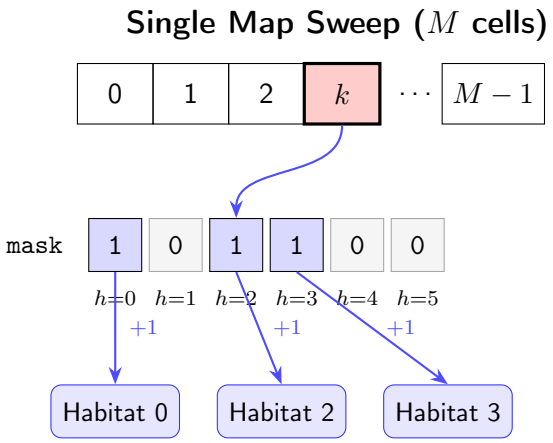

Figure 1.4: Map-Centric update for a single invaded cell k. One $\mathcal{U}(0,1)$ draw is taken for the cell; on invasion ($x_k \le p_k$) its 64-bit presence mask is decoded and every habitat occupying the cell (bits set to 1) has its invaded-cell counter incremented in the same branchless pass. Here cell k belongs to habitats 0, 2 and 3.

Rather than branching on whether each habitat is present—which would introduce thread divergence within a warp—the kernel unconditionally extracts bit h and adds it to the corresponding counter (Figure 1.4). The increment is therefore 1 for a present habitat and 0 otherwise, with no divergent control flow. After the sweep, each habitat's destruction is assessed with the same strict-threshold rule of Section 1.1.2: the run is a destruction event for habitat h if and only if `run_invaded[h]`/`hab_total_cells[h]` $> \theta_h$.

Two-Dimensional Local Reduction. The work-group then collapses the per-work-item destruction counts for *all* habitats at once. The local `scratch` buffer is laid out as `scratch`$[h \cdot L + $`lid`$]$—one row per habitat, one column per lane—and the usual power-of-2 tree reduction runs over the lane axis for every habitat row. This transposed layout is deliberately bank-conflict free: consecutive lanes within a habitat row map to consecutive memory banks (stride 1), so the threads of a warp never contend for the same bank. Finally, `lid 0` writes each habitat's group total to `partial`$[h \cdot n_{wg} +$ `group_id`$]$, and the host closes the reduction by summing the n_{wg} partials of each habitat.

1.6.2 Trade-offs

The map-centric design trades the redundant sampling of overlapping habitats for a heavier per-cell update, and whether this trade is favourable depends entirely on how much the habitats actually overlap.

Benefit (RNG reuse). The single map sweep amortises the cost of random number generation across all habitats sharing a cell. The deeper the geographic overlap, the larger the saving, since the number of draws is bounded by the union footprint M rather than by the sum $\sum_h N_c^{(h)}$ of the individual habitat sizes.

Cost (branchless inner loop). The very feature that removes divergence also removes the ability to skip absent habitats: the inner loop performs $H =$ `num_habitats` iterations for *every* invaded cell, regardless of how many habitats truly occupy it. The accumulation cost thus scales with the batch size, not with the actual overlap depth of the cell.

Cost (register pressure). Each work-item maintains two private `int[MAX_BATCH_SIZE]` arrays (`run_invaded` and `private_destroyed`), i.e. up to 512 bytes of private state. This footprint tends to spill registers, lowering the number of concurrent wavefronts and hence the hardware occupancy.

Consequently the map-centric kernel only outperforms the habitat-centric baselines once the average overlap depth is high enough for the RNG-reuse saving to overcome both the H-wide inner loop and the increased register pressure. With little or no overlap it is dominated by the baseline. A quantitative comparison is deferred to the benchmark in Section 1.7.

1.6.3 Algorithmic Complexity

The complexity of the map-centric kernel is governed by the number of map cells M (`n_map_cells`) and the number of habitats per batch H (`num_habitats`, with $H \leq$ `MAX_BATCH_SIZE` $= 64$), in addition to the usual parameters R, P and L.

Time Complexity. The time complexity is again divided into the two principal components:

Sampling and Monte Carlo Run: each work-item processes R/P runs through the sliding window loop. Within a run it sweeps the M map cells, and in the worst case (every cell invaded) the branchless inner loop executes H iterations per cell, yielding $\mathcal{O}(M \cdot H)$ work per run. The per-habitat destruction assessment adds a further $\mathcal{O}(H)$ that is dominated by the sweep. The sampling phase therefore costs

$$\mathcal{O}\left(\frac{R}{P} \cdot M \cdot H\right)$$

Reduction: the two-dimensional tree reduction operates over the L lanes of the workgroup for each of the H habitat rows, resulting in $\mathcal{O}(H \cdot \log_2 L)$.

The overall time complexity is then

$$T(M, H, R, P, L) = \mathcal{O}\left(\frac{R}{P} \cdot M \cdot H + H \cdot \log_2 L\right) \tag{1.9}$$

Since H is bounded by the hardware constant `MAX_BATCH_SIZE` $= 64$, this is asymptotically $\mathcal{O}\big(\frac{R}{P} \cdot M + \log_2 L\big)$. The practical significance, however, lies precisely in the constants: M can be substantially smaller than $\sum_h N_c^{(h)}$ when habitats overlap (the RNG-reuse benefit), while the H factor on the inner loop is the per-cell penalty discussed in Section 1.6.2.

Space Complexity. The space complexity reflects the cost of carrying all habitats through a single kernel invocation:

- **Global Memory**: the kernel reads the full-map `p_vec` ($\mathcal{O}(M)$ floats) and `habitat_mask` ($\mathcal{O}(M)$ `ulong`s), plus the per-habitat `hab_total_cells` and `hab_thresholds` ($\mathcal{O}(H)$ each). In output it writes the `partial` array of H rows by $n_{wg} = P/L$ columns. The total global memory complexity is therefore $\mathcal{O}(M + H \cdot P/L)$.

- **Local Memory**: the dynamic `scratch` buffer holds one column per lane for each habitat row, i.e. $L \cdot H$ integers. The local memory complexity is thus $\mathcal{O}(L \cdot H)$ per work-group—an H-fold expansion over the scalar reduction, which further constrains the achievable occupancy.

- **Private Memory**: each work-item allocates the two `int[MAX_BATCH_SIZE]` accumulators (`run_invaded` and `private_destroyed`) together with the RNG state and a few loop counters. The private footprint therefore scales as $\mathcal{O}(H)$ per work-item (bounded by the constant batch size), as opposed to the $\mathcal{O}(1)$ of the scalar kernel.

1.7 TODO: Benchmark

DUMMY CIT [1]

Note e appunti.

Cose da vedere:

- Il numero di work-items e work-groups lanciati per ogni kernel, e come questo si relaziona con il numero totale di simulazioni R e la dimensione del lavoro locale L. Sopratutto per fare vedere la differenza tra due barriere e ping pong.

Appendix A

Library Architecture

(Appendix placeholder — to be expanded.)

Bibliography

[1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, apr 2009.